

Distributed Architectural Patterns and Evolution of Remoting Technologies

Distributed Application Development in the Cloud – NET 031

Prepared By: Supun Kandaudahewa
supun.kandaudahewa@humber.ca



WE ARE

HUMBER

Agenda

- Distributed Systems
- Why Remote?
- Client-Server Architecture
- Peer-to-Peer (P2P)
- Service Oriented Architecture (SOA)
- Microservices Architecture
- VM's and Containers
- Message Exchange Patterns (MEP)
 - SOAP
 - REST
 - gRPC
- 3 Fallacies of Distributed Computing

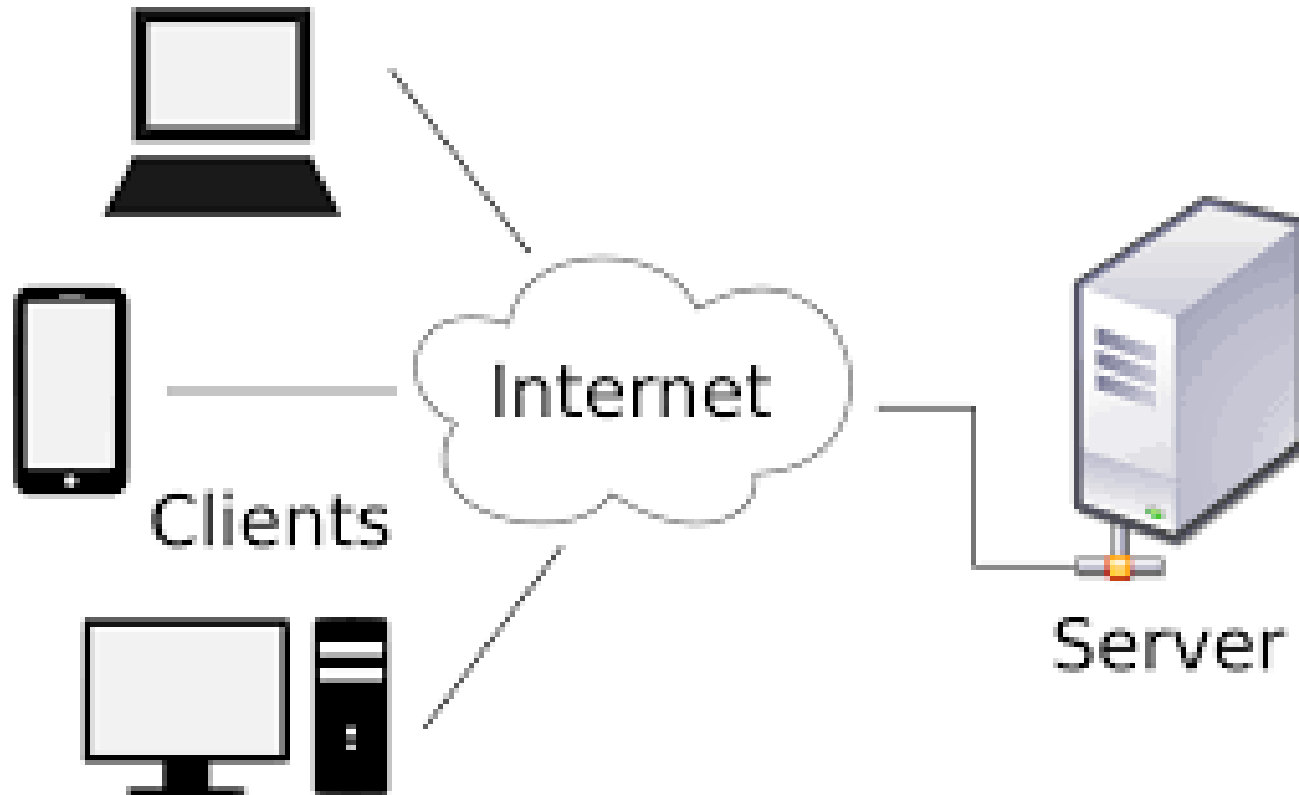
Distributed Systems

- A distributed system is a system whose components are located on different networked computers.
- When components are apart, we need **effective communication** and **efficient coordination** among them to get the job done.
- But there are challenges:
 - Maintaining concurrency.
 - Overcoming the lack of a global clock.
 - Managing the failures.
- Examples:
 - SOA based systems (including micro services)
 - Peer-to-peer applications
 - Serverless architecture (new kid in town!)

Why Remote?

- Remoting was typically used for accessing a server's resources at the beginning of the client/server era.
- Every file server or database is an implementation of some tactics that allow code to be executed remotely.
- The building of distributed applications makes it easy to distribute business logic among diverse machines to improve **performance, maintainability and scalability.**

Client Server Architecture



https://en.wikipedia.org/wiki/Client%E2%80%93server_model

Client Server Architecture Contd.

- Client-Server architecture allows us to partition tasks or workloads between the providers of a resource or service and requesters of the service.
- The providers are called as **Servers** while the requesters are called as **Clients**. (This is the simplest definition)
- Client → Request things from the server.
- Server → Responds to the request and send results if needed.

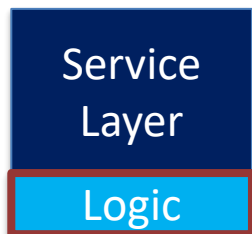
The need for cross-platform communication

- Enterprises usually consist of a combination of different platforms: Windows, Linux
- And programming languages: C#, C++, Java, PHP
- Remoting technologies like REST and SOAP are an absolute necessity in large-scale enterprise application integration.

Advantages of having Centralized Business Logic

- A key scenario for implementing remoting technology is the concentration of business logic on a central server.
- Simplifies the maintainability and operability of large-scale applications.
- We only have to update one single server rather a number of users separately.
- When a centralized architecture shares different applications, it reduces programming labor.
- Let's see this in a more pictorial way.

Advantages of having Centralized Business Logic Contd.



The logic is scattered. In case of a change multiple places needs to be updated.

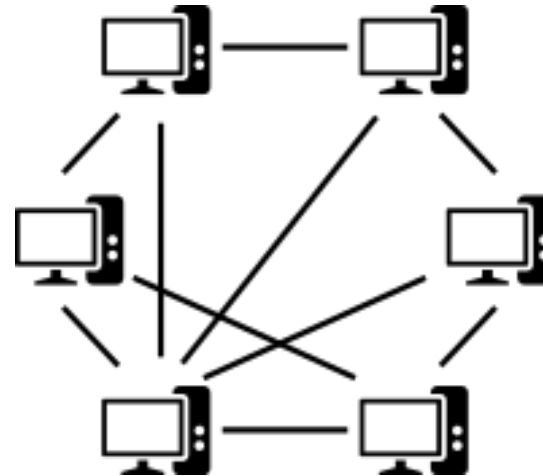


Business Logic is concentrated. And it can be easily maintained



Peer to Peer (P2P)

- A **peer-to-peer (P2P)** network in which interconnected nodes ("peers") share resources amongst each other without the use of a centralized administrative system.
- Peers are both suppliers and consumers of resources. This is the main difference between Client-Server and Peer-To-Peer.

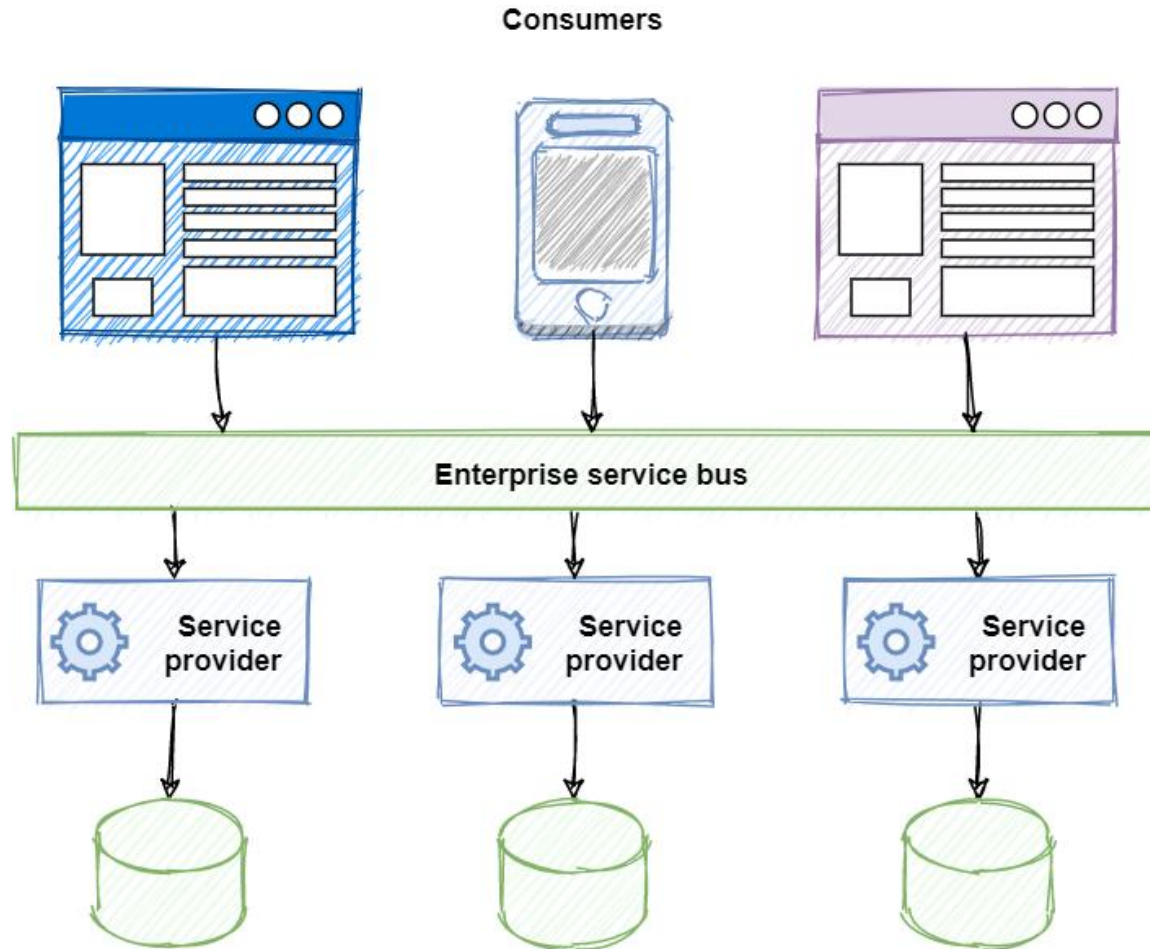


Service Oriented Architecture (SOA)

- As the number of services increased point-to-point became difficult to maintain (e.g., many connection strings, updates when the topology changed, translation issues).
- SOA used the concept of a **Service-Bus** to interconnect systems via a hub type architecture.
- Each system could build server side and client-side implementations without worrying about the protocols of the connecting systems.

SOA Contd.

<https://learn.microsoft.com/en-us/dotnet/architecture/dapr-for-net-developers/the-world-is-distributed>

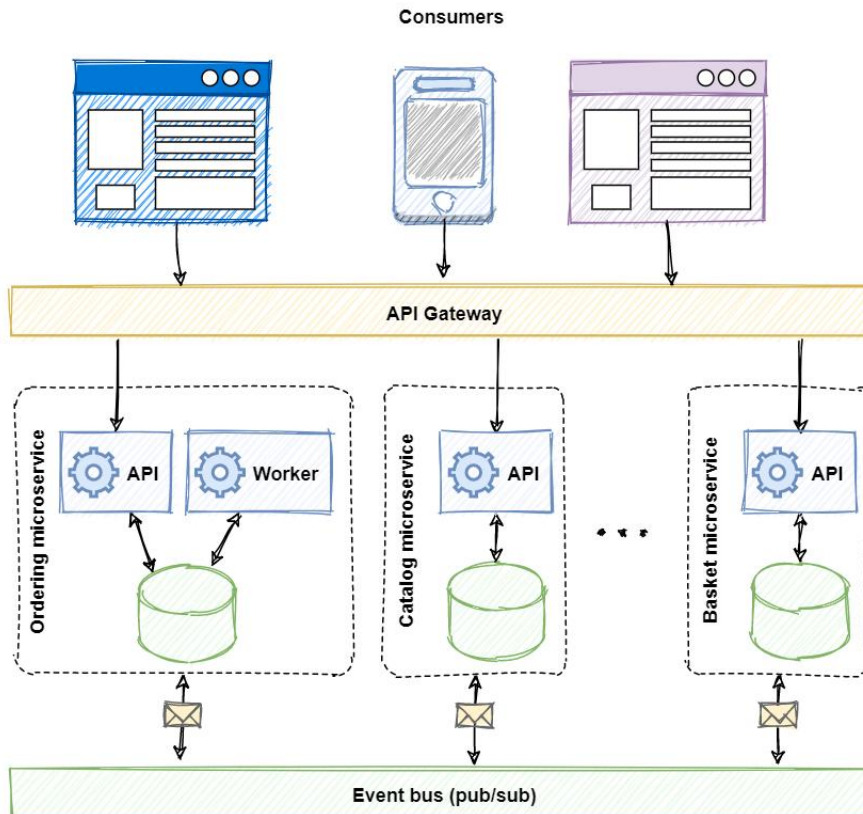


The role of ESB

- An ESB removes the direct connection between service consumer and provider by:
 - Decoupling technical aspects of service interactions.
 - Integrating and managing services in the enterprise.
 - Map service requests from one protocol and address to another format.
 - Aggregate or disaggregated service requests and responses.
- Drawback : Bottlenecks in ESB (complexity, resistance to change)

Microservices Architecture

- A more fine-grained version of SOA.



<https://learn.microsoft.com/en-us/dotnet/architecture/dapr-for-net-developers/the-world-is-distributed>

Characteristics of Microservices

- Microservice is a type of service-oriented software architecture that focuses on building a series of autonomous components that make up an app.
- Microservice apps consist of multiple independent components that are glued together with APIs.
- The **microservices approach focuses mainly on business priorities** and capabilities, not on the technologies. → It is very easier to map and maintain the domain.
- Superior agility and scalability.

Discussion: What are the downsides of Microservices?

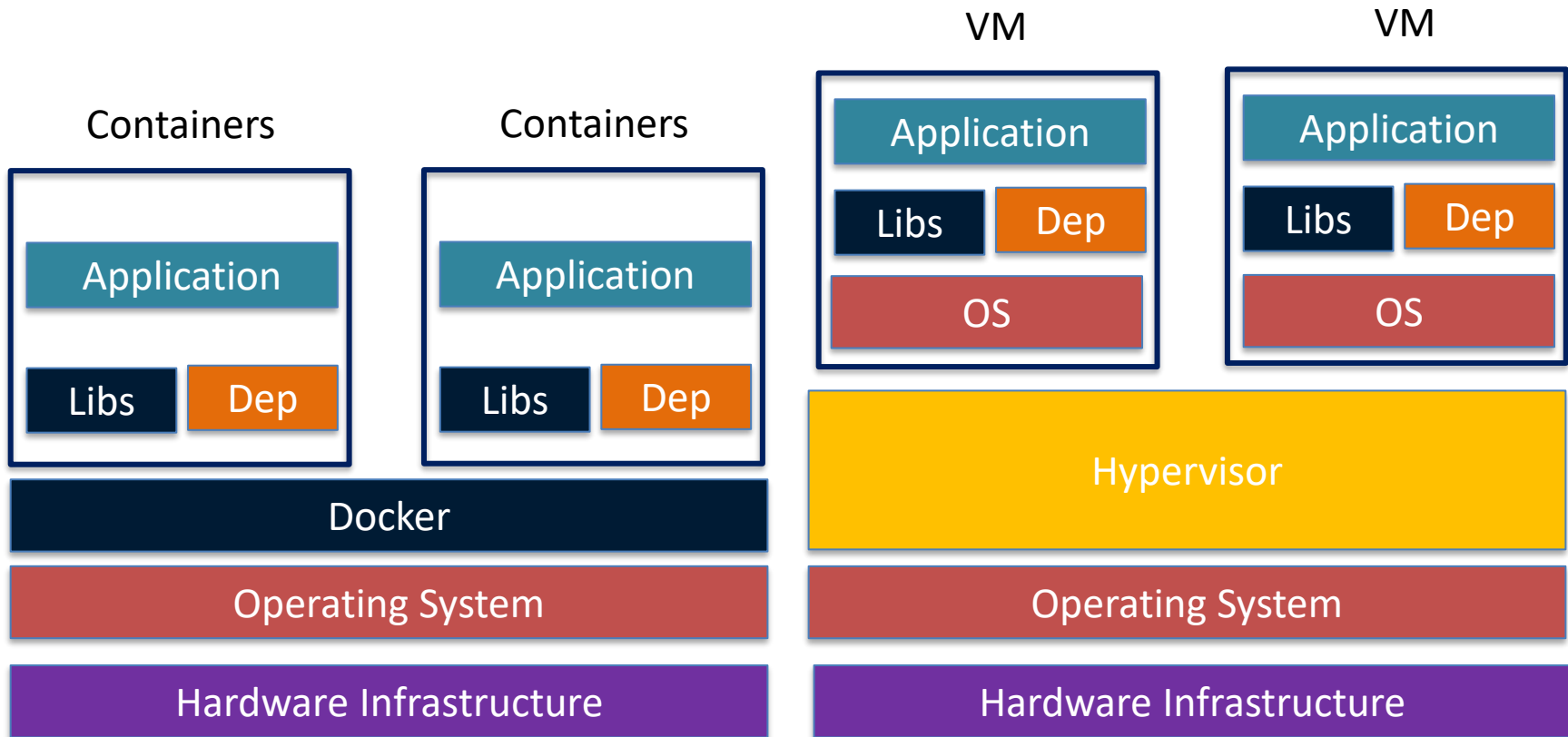
Virtual Machines (VM)

- With the rise of global-scale applications companies needed to build distributed systems that went beyond single data centers.
- These applications needed to span multiple geographies.
- Architects started to design systems that acted like single systems but spanned more than one physical system.
- Virtual machines allowed applications to scale relatively simply.
- Creating a VM was much more efficient than installing a unit of physical hardware into a data-center.
- With VMs the operating system saw itself as running on its own hardware when in reality it maybe running on a server that is hosting many individual VMs.

Containers

- Containers virtualize only the application and its direct dependencies.
- The OS is shared.
- Extremely fast to boot and start serving client requests.
- Immutable – containers can be started and stopped and their configuration data(or state) is stored outside the container so it can be restored anytime.

Containers Vs Virtual Machines



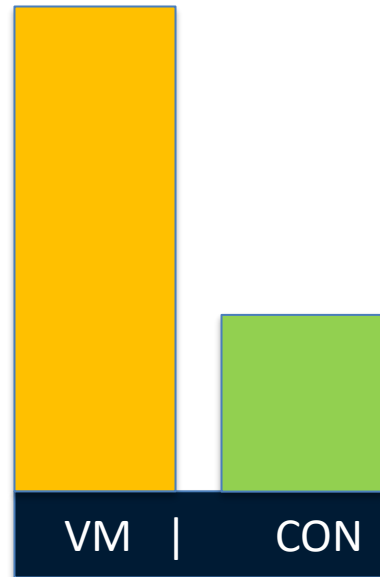
Containers Vs Virtual Machines Contd.



Resource Utilization



Storage requirements



Boot time



Licensing cost

Message Exchange Patterns (MEP)

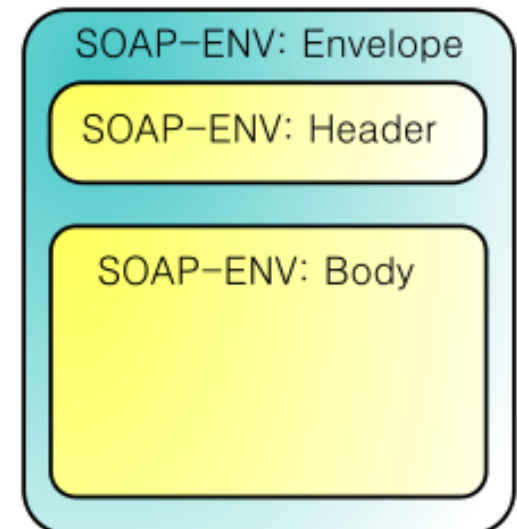
- Messaging Patterns is an architectural pattern which describes how **two different parts of an application**, or **different systems connect and communicate** with each other.
- There are many aspects to the concept of messaging which can be divided in the following categories:
 - Hardware device messaging (telecommunications, computer networking, IoT, etc.)
 - Software data exchange (the different data exchange formats and software capabilities of such data exchange)

https://en.wikipedia.org/wiki/Messaging_pattern

Simple Object Access Protocol (SOAP)

- SOAP is a messaging protocol specification for exchanging structured information in the implementation of web services in computer networks.
- XML Based.
- Extensible – Use cases are extensible.
- Neutral - Can operate over any protocol such as HTTP, SMTP, TCP, UDP.
- Independent –Allows any programming model.

<https://en.wikipedia.org/wiki/SOAP>



REpresentational State Transfer (REST)

- Defines a set of constraints to be used for creating web services.
- Roy Fielding defined REST in his 2000 PhD dissertation at UC Irvine.

Architectural Styles and the Design of Network-based Software Architectures

- He developed the REST architectural style in parallel with HTTP 1.1 of 1996–1999, based on the existing design of HTTP 1.0.
- Based on HTTP and REST and HTTP(S) cannot be separated
- Verb based CRUD operations (GET/POST/PUT/DELETE/PATCH)

gRPC Remote Procedure Call Framework



gRPC Remote Procedure Call Framework

simple and idiomatic

efficient and scaleable

interoperable and extensible

baked in authentication support

<https://www.capitalone.com/tech/software-engineering/grpc-framework-for-microservices-communication/>

gRPC Remote Procedure Call Framework Contd.

- A remote procedure implementation initially developed at Google.
- Uses HTTP/2 and Protocol Buffers as the service description language.
- A binary framing protocol for data transport - unlike HTTP 1.1, which is text based.
- Multiplexing support for sending multiple parallel requests over the same connection - HTTP 1.1 limits processing to one request/response message at a time.
- Header compression that reduces network usage.

Fallacies of Distributed Computing

- The network is reliable.
- Latency is zero.
- Bandwidth is infinite.
- The network is secure.
- Topology doesn't change.
- There is one administrator.
- Transport cost is zero.
- The network is homogeneous.

Additional Resources

- <https://dotnet.microsoft.com/en-us/download/e-book/microservices-architecture/pdf>

THANK YOU.



WE ARE

HUMBER